

CINEVAULT RENTALS

SQL DATABASE ANALYST ASSESSMENT

Database: sakila

Section A — Schema Design & DDL

Q1. New Franchise Rollout

Question:

CineVault is opening a new Employee Performance tracking module, separate from the existing staff table.

a) Create a table cv_employee_perf with: perf_id (INT, PK, AUTO_INCREMENT), staff_id (INT), review_month (DATE), films_processed (INT), rating (DECIMAL(3,1)), remarks (VARCHAR(255)).

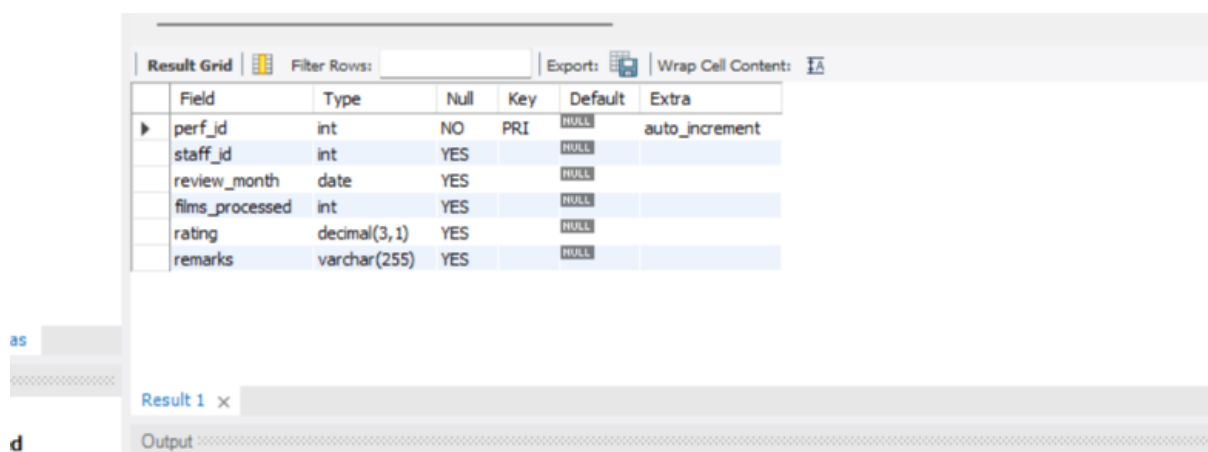
SQL Query:

```
CREATE TABLE cv_employee_perf (  
  perf_id INT PRIMARY KEY AUTO_INCREMENT,  
  staff_id INT,  
  review_month DATE,  
  films_processed INT,  
  rating DECIMAL(3,1),  
  remarks VARCHAR(255)  
);
```

EXPLANATION:

This query creates a new table named cv_employee_perf to store employee performance details. perf_id is the primary key and automatically generates a unique ID for each record. The table also stores the staff ID, review month, number of films processed, performance rating, and remarks.

Output:



The screenshot shows a database management tool interface. At the top, there's a toolbar with 'Result Grid', 'Filter Rows', 'Exports', and 'Wrap Cell Contents'. Below this is a table representing the schema of the cv_employee_perf table. The table has columns: Field, Type, Null, Key, Default, and Extra. The rows are: perf_id (int, NO, PRI, NULL, auto_increment), staff_id (int, YES, NULL, NULL), review_month (date, YES, NULL, NULL), films_processed (int, YES, NULL, NULL), rating (decimal(3,1), YES, NULL, NULL), and remarks (varchar(255), YES, NULL, NULL). Below the table, there's a section labeled 'Result 1' with a close button 'x'. At the bottom, there's a section labeled 'Output'.

Field	Type	Null	Key	Default	Extra
perf_id	int	NO	PRI	NULL	auto_increment
staff_id	int	YES		NULL	
review_month	date	YES		NULL	
films_processed	int	YES		NULL	
rating	decimal(3,1)	YES		NULL	
remarks	varchar(255)	YES		NULL	

b) Add a FOREIGN KEY on staff_id referencing staff(staff_id).

SQL Query:

— Datatype compatibility:

```
ALTER TABLE cv_employee_perf  
MODIFY staff_id TINYINT UNSIGNED;
```

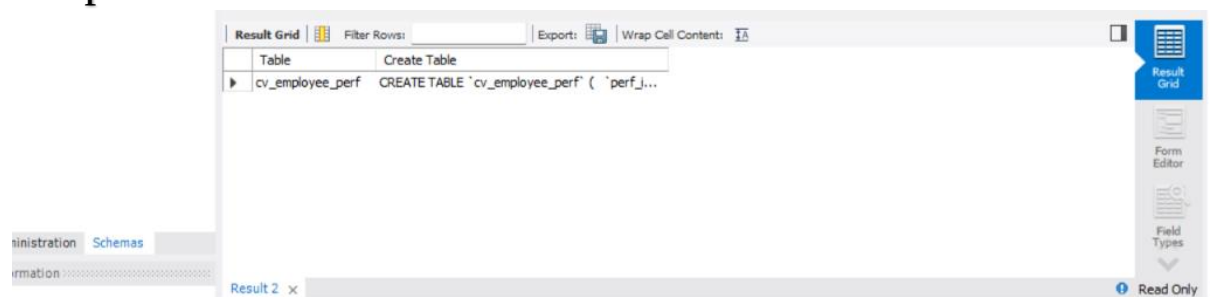
— Add Foreign Key:

```
ALTER TABLE cv_employee_perf  
ADD CONSTRAINT fk_cv_employee_perf_staff  
FOREIGN KEY (staff_id)  
REFERENCES staff(staff_id);  
SHOW CREATE TABLE cv_employee_perf;
```

EXPLANATION:

This query adds a **foreign key constraint** to the staff_id column in cv_employee_perf. It links staff_id to the staff_id column of the staff table, ensuring that the employee performance record refers to a valid staff member.

Output:



c) Management realizes rating should never be NULL and should default to 3.0. Alter the table accordingly.

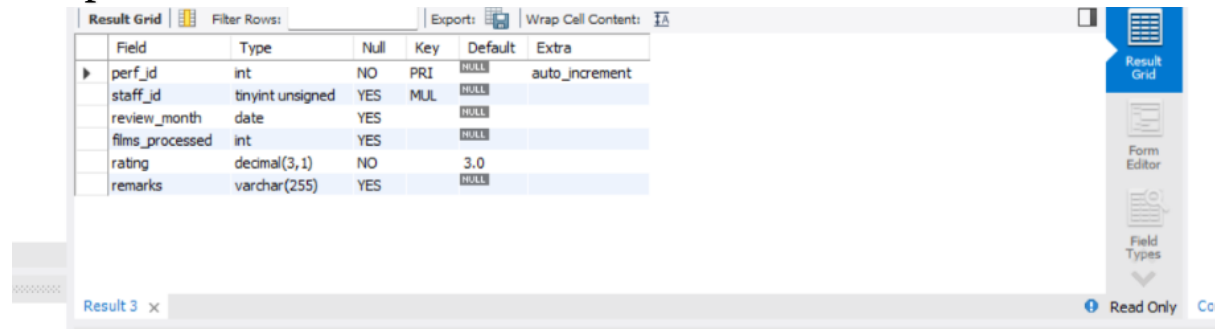
SQL Query:

```
ALTER TABLE cv_employee_perf  
MODIFY rating DECIMAL(3,1) NOT NULL DEFAULT 3.0;  
DESCRIBE cv_employee_perf;
```

EXPLANATION:

This query modifies the rating column so that it **cannot contain NULL values**. It also sets **3.0 as the default rating** when no rating is provided while inserting a record.

Output:



Field	Type	Null	Key	Default	Extra
perf_id	int	NO	PRI	NULL	auto_increment
staff_id	tinyint unsigned	YES	MUL	NULL	
review_month	date	YES		NULL	
films_processed	int	YES		NULL	
rating	decimal(3,1)	NO		3.0	
remarks	varchar(255)	YES		NULL	

d) The remarks column is rarely used and wastes space in this design — rename it to notes and change its data type to TEXT in the same statement (or two, if your MySQL version needs it).

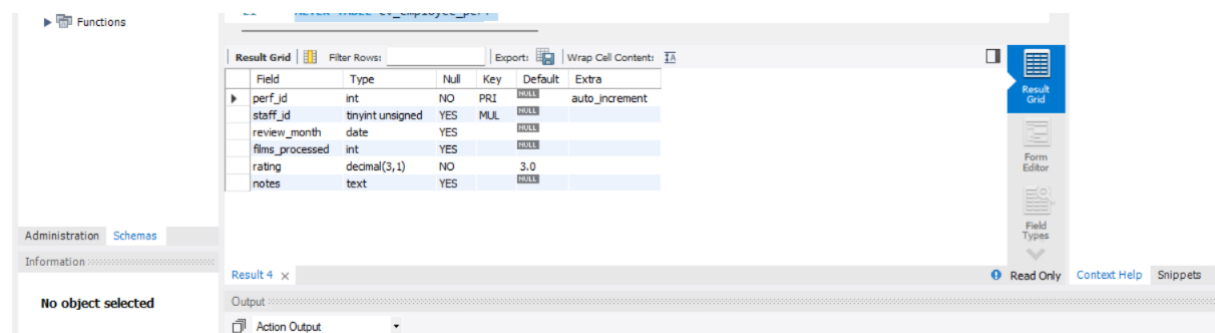
SQL Query:

```
ALTER TABLE cv_employee_perf  
CHANGE remarks notes TEXT;
```

EXPLANATION:

This query changes the remarks column name to **notes** and changes its data type from **VARCHAR(255) to TEXT**. This allows the column to store longer text when required.

Output:



Field	Type	Null	Key	Default	Extra
perf_id	int	NO	PRI	NULL	auto_increment
staff_id	tinyint unsigned	YES	MUL	NULL	
review_month	date	YES		NULL	
films_processed	int	YES		NULL	
rating	decimal(3,1)	NO		3.0	
notes	text	YES		NULL	

Q2. Data Type Conversion Issue

Finance reports that the rental_rate in the film table (currently DECIMAL(4,2)) is too restrictive for a new premium pricing tier going up to 999.99.

- a) Write the ALTER TABLE statement to widen rental_rate to DECIMAL(6,2) without losing existing data.

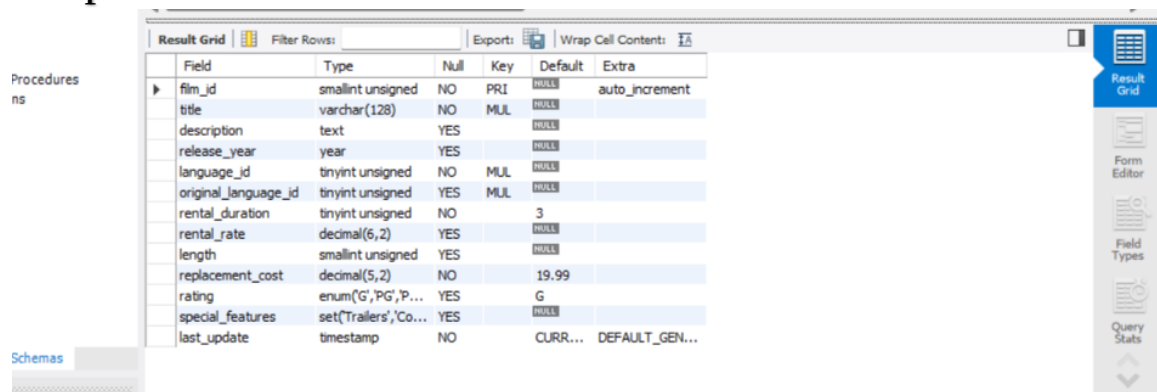
SQL Query:

```
ALTER TABLE film
MODIFY rental_rate DECIMAL(6,2);
DESCRIBE film;
```

EXPLANATION:

This query modifies the rental_rate column in the film table from DECIMAL(4,2) to DECIMAL(6,2). This increases the number of digits that can be stored while keeping **2 digits after the decimal point**.

Output:



Field	Type	Null	Key	Default	Extra
film_id	smallint unsigned	NO	PRI	NULL	auto_increment
title	varchar(128)	NO	MUL	NULL	
description	text	YES		NULL	
release_year	year	YES		NULL	
language_id	tinyint unsigned	NO	MUL	NULL	
original_language_id	tinyint unsigned	YES	MUL	NULL	
rental_duration	tinyint unsigned	NO		3	
rental_rate	decimal(6,2)	YES		NULL	
length	smallint unsigned	YES		NULL	
replacement_cost	decimal(5,2)	NO		19.99	
rating	enum('G','PG','P...)	YES		G	
special_features	set('Trailers','Co...)	YES		NULL	
last_update	timestamp	NO		CURR...	DEFAULT_GEN...

- b) Write a query to convert and display the payment_date (DATETIME) from the payment table as just a DATE value (no time component) using a conversion function — do not alter the actual table

SQL Query:

```
SELECT payment_id,
       customer_id,
```

```

amount,
CAST(payment_date AS DATE) AS payment_date
FROM payment;

```

EXPLANATION:

This query uses CAST() to convert the payment_date value from **DATETIME** to **DATE** for display. It shows only the date portion and **does not modify the original column or table structure**.

Output:

The screenshot shows a database management tool interface. The 'Result Grid' displays a table with 4 columns: payment_id, customer_id, amount, and payment_date. The 'Action Output' pane at the bottom shows a successful execution of an ALTER TABLE statement.

payment_id	customer_id	amount	payment_date
1	1	2.99	2005-05-25
2	1	0.99	2005-05-28
3	1	5.99	2005-06-15
4	1	0.99	2005-06-15
5	1	9.99	2005-06-15
6	1	4.99	2005-06-16
7	1	4.99	2005-06-18
8	1	0.99	2005-06-18
9	1	3.99	2005-06-21
10	1	5.99	2005-07-08
11	1	5.99	2005-07-08
12	1	4.99	2005-07-09
13	1	4.99	2005-07-09
14	1	7.99	2005-07-11
15	1	2.99	2005-07-27

#	Time	Action	Message
14	19:28:45	ALTER TABLE film MODIFY rental_rate DECIMAL(6,2)	1000 row(s) affected Records: 1000 Duplicates: 0 Warnings: 0

Q3. Decommissioning & Cleanup

The company is discontinuing the cv_employee_perf pilot after testing.

- Write a query to remove all rows from cv_employee_perf but keep the table structure intact for future use.

SQL Query:

```

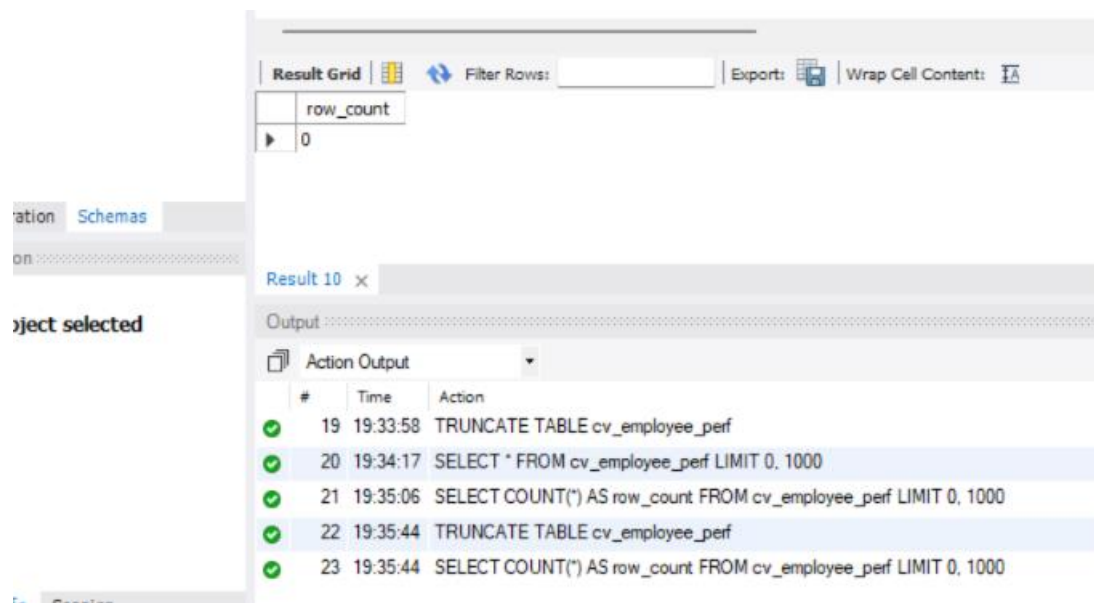
TRUNCATE TABLE cv_employee_perf;
SELECT COUNT(*) AS row_count
FROM cv_employee_perf;

```

EXPLANATION:

This query uses TRUNCATE TABLE to **remove all records** from cv_employee_perf while keeping the **table structure, columns, and constraints** intact. The table can therefore be reused later if required.

Output:



b) Six months later, management confirms the module is cancelled permanently. Write the query to remove the table entirely.

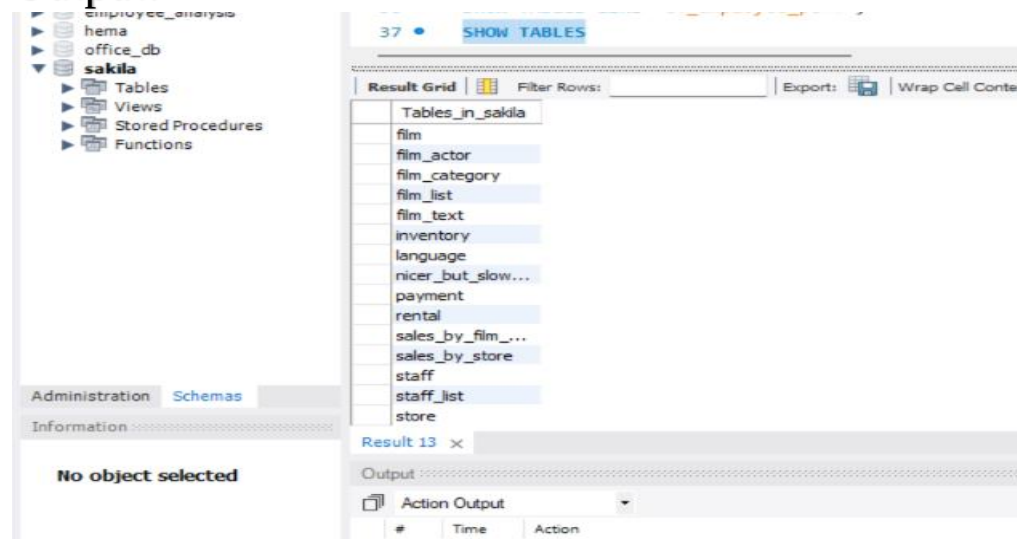
SQL Query:

```
DROP TABLE cv_employee_perf;
```

EXPLANATION:

This query uses DROP TABLE to **permanently delete the cv_employee_perf table** along with all its data and structure. After this command, the table will no longer exist in the database.

Output:



C) In 2–3 lines, explain why you'd choose TRUNCATE for (a) and DROP for (b) instead of using DELETE for both.

EXPLANATION:

TRUNCATE is used in Q3(a) because it removes all rows while keeping the table structure intact for future use. DROP is used in Q3(b) because the module is permanently cancelled and the entire table is no longer required. DELETE is not preferred for removing all rows because TRUNCATE is more efficient for clearing the complete table.

Q4. Normalization Check

A new intern proposes storing customer_name, customer_email, film_title, and rental_date all in one flat table called rental_log_flat for “simplicity.”

a) Identify which Normal Form this design violates and explain briefly (2–3 lines) why storing repeating customer/film details in every rental row is a problem.

EXPLANATION:

The flat table design violates **Third Normal Form (3NF)** because customer and film details depend on their respective entities rather than directly on the rental record. Repeating customer names, emails, and film titles in every rental row causes data redundancy and can lead to update, insertion, and deletion anomalies.

b) Using the existing Sakila schema (customer, film, rental, inventory) as your reference, briefly explain (2–3 lines) how Sakila already avoids this problem — name the tables/keys involved

EXPLANATION:

Sakila separates customer, film, and rental information into different tables instead of repeating the details in every rental record. The rental table uses customer_id and inventory_id as foreign keys, while inventory connects to film through film_id, reducing data redundancy and maintaining data consistency.

Section B — Data Manipulation

Q5. Onboarding New Staff

Operations hired a new store employee. Insert a new row into staff for a person of your choice, ensuring you don't break any NOT NULL or FOREIGN KEY constraints (check the table structure first). List the columns explicitly in your INSERT statement.

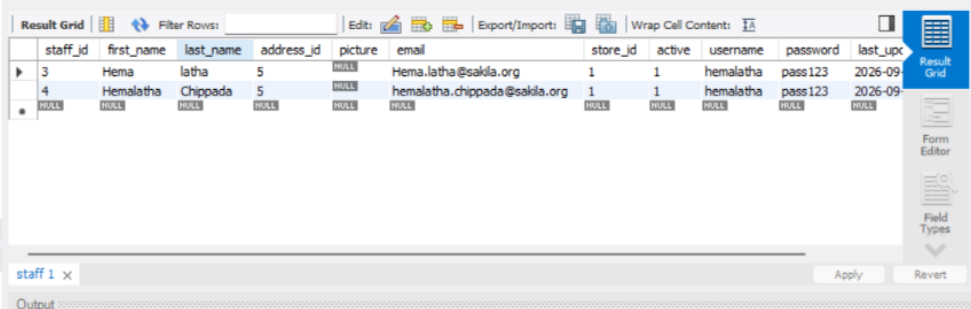
SQL Query:

```
INSERT INTO staff
    (first_name, last_name,
     address_id, email, store_id,
     active, username, password)
VALUES
    ('Hemalatha', 'chippada', 5,
     'hemalatha.chippada@sakila.org',
     1, 1, 'hemalatha', 'pass123');
SELECT *
FROM staff
WHERE username = 'hemalatha';
```

EXPLANATION:

The INSERT INTO statement adds a new staff member named **Hema Chippada** with the required details. The SELECT statement then verifies the inserted record by searching for the username hema.chippada.

Output:



staff_id	first_name	last_name	address_id	picture	email	store_id	active	username	password	last_update
3	Hema	latha	5	NULL	Hema.latha@sakila.org	1	1	hemalatha	pass123	2026-09-26 10:26:09
4	Hemalatha	Chippada	5	NULL	hemalatha.chippada@sakila.org	1	1	hemalatha	pass123	2026-09-26 10:26:09

Q6. Price Correction — Bulk Update

Finance discovered that all films with rating = 'PG' were under-priced during a system migration.

a) Write an UPDATE query to increase the rental_rate of all 'PG' rated films by 15%, rounded to 2 decimal places

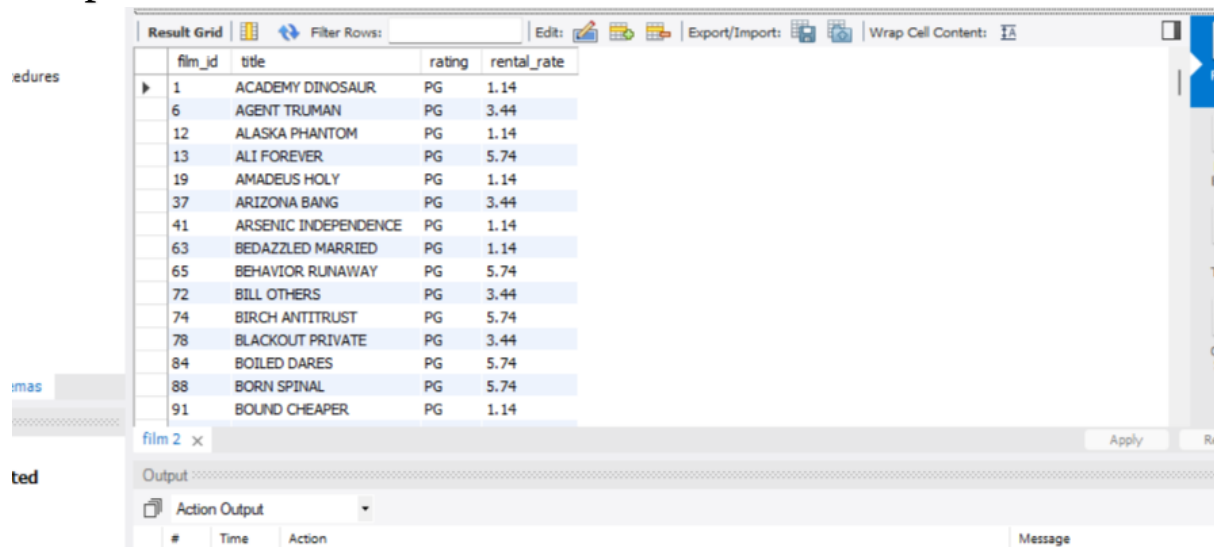
SQL Query:

```
UPDATE film
SET rental_rate = ROUND(rental_rate * 1.15, 2)
WHERE rating = 'PG' AND film_id > 0;
```

EXPLANATION:

The UPDATE statement increases the rental rate of all PG-rated films by **15%** using ROUND() to keep 2 decimal places. SQL_SAFE_UPDATES is temporarily disabled to allow the bulk update and then enabled again. The final SELECT verifies the updated PG film rental rates.

Output:



The screenshot shows a database application interface. At the top, there's a toolbar with options like 'Result Grid', 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. Below this is a table with columns 'film_id', 'title', 'rating', and 'rental_rate'. The table contains 15 rows of data, all with a 'PG' rating. The rental rates are: 1.14, 3.44, 1.14, 5.74, 1.14, 3.44, 1.14, 5.74, 3.44, 5.74, 3.44, 5.74, 5.74, 3.44, and 1.14. Below the table, there's a section labeled 'film 2' with an 'Apply' button. At the bottom, there's an 'Output' section with a dropdown menu set to 'Action Output' and a table with columns '#', 'Time', 'Action', and 'Message'.

film_id	title	rating	rental_rate
1	ACADEMY DINOSAUR	PG	1.14
6	AGENT TRUMAN	PG	3.44
12	ALASKA PHANTOM	PG	1.14
13	ALI FOREVER	PG	5.74
19	AMADEUS HOLY	PG	1.14
37	ARIZONA BANG	PG	3.44
41	ARSENIC INDEPENDENCE	PG	1.14
63	BEDAZZLED MARRIED	PG	1.14
65	BEHAVIOR RUNAWAY	PG	5.74
72	BILL OTHERS	PG	3.44
74	BIRCH ANTITRUST	PG	5.74
78	BLACKOUT PRIVATE	PG	3.44
84	BOILED DARES	PG	5.74
88	BORN SPINAL	PG	5.74
91	BOUND CHEAPER	PG	1.14

b) Finance now wants to cap the replacement_cost at 25.00 for any film currently priced above that — write the query.

SQL Query:

```
UPDATE film SET replacement_cost = 25.00
WHERE replacement_cost > 25.00
AND film_id > 0;
SELECT film_id, title, replacement_cost
FROM film
WHERE replacement_cost = 25;
```

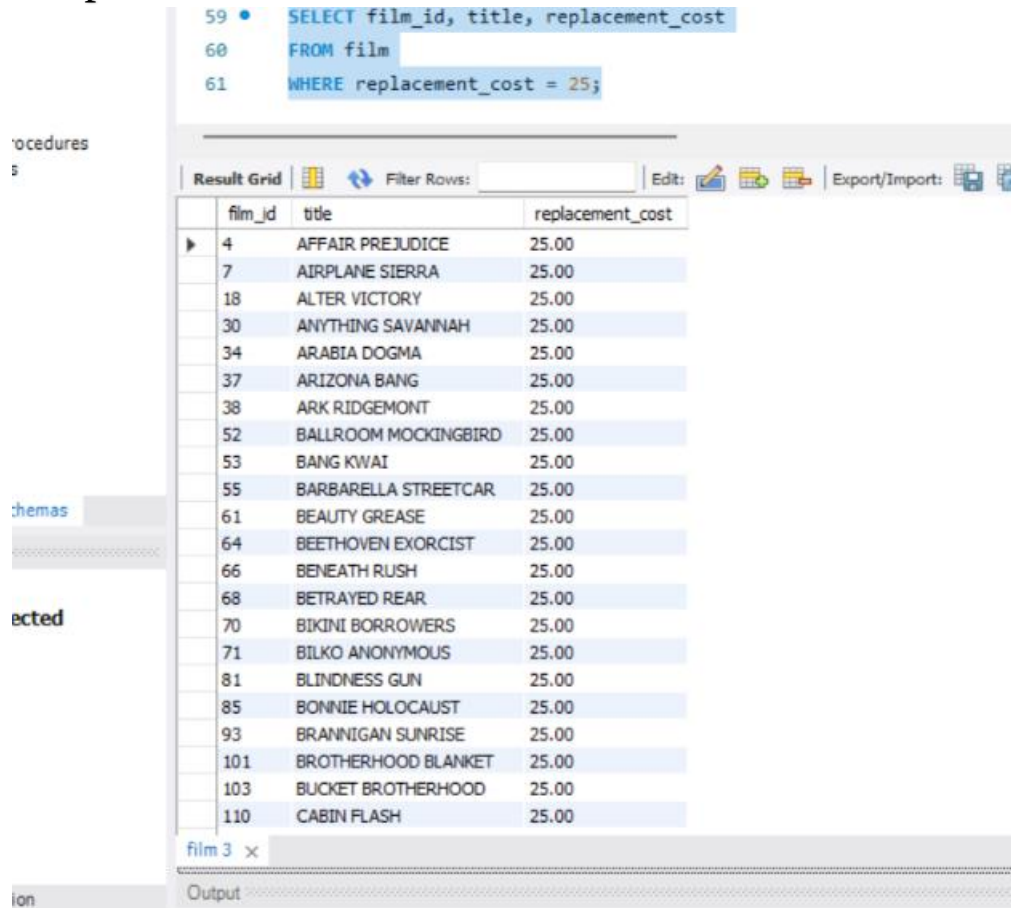
EXPLANATION:

The UPDATE statement changes the replacement_cost to **25.00** for all films whose current replacement cost is greater than 25.00. Safe Update Mode is

temporarily disabled to allow the bulk update, and the final SELECT verifies the updated records.

Output:

```
59 • SELECT film_id, title, replacement_cost
60 FROM film
61 WHERE replacement_cost = 25;
```



The screenshot shows a database interface with a SQL query editor at the top and a 'Result Grid' below it. The query is: `SELECT film_id, title, replacement_cost FROM film WHERE replacement_cost = 25;`. The 'Result Grid' displays a table with three columns: `film_id`, `title`, and `replacement_cost`. The table contains 20 rows of data, all with a `replacement_cost` of 25.00. The interface includes a 'Filter Rows' field, an 'Edit' button, and an 'Export/Import' button. The bottom of the interface shows a tab for 'film 3' and an 'Output' section.

film_id	title	replacement_cost
4	AFFAIR PREJUDICE	25.00
7	AIRPLANE SIERRA	25.00
18	ALTER VICTORY	25.00
30	ANYTHING SAVANNAH	25.00
34	ARABIA DOGMA	25.00
37	ARIZONA BANG	25.00
38	ARK RIDGEMONT	25.00
52	BALLROOM MOCKINGBIRD	25.00
53	BANG KWAI	25.00
55	BARBARELLA STREETCAR	25.00
61	BEAUTY GREASE	25.00
64	BEETHOVEN EXORCIST	25.00
66	BENEATH RUSH	25.00
68	BETRAYED REAR	25.00
70	BIKINI BORROWERS	25.00
71	BILKO ANONYMOUS	25.00
81	BLINDNESS GUN	25.00
85	BONNIE HOLOCAUST	25.00
93	BRANNIGAN SUNRISE	25.00
101	BROTHERHOOD BLANKET	25.00
103	BUCKET BROTHERHOOD	25.00
110	CABIN FLASH	25.00

Q7. Data Cleanup — Conditional Delete

Operations wants to purge test/junk data.

a) Write a query to delete rows from `cv_employee_perf` where `films_processed = 0` and `rating < 2.0`.

SQL Query:

```
DELETE FROM cv_employee_perf
```

```
WHERE films_processed = 0
```

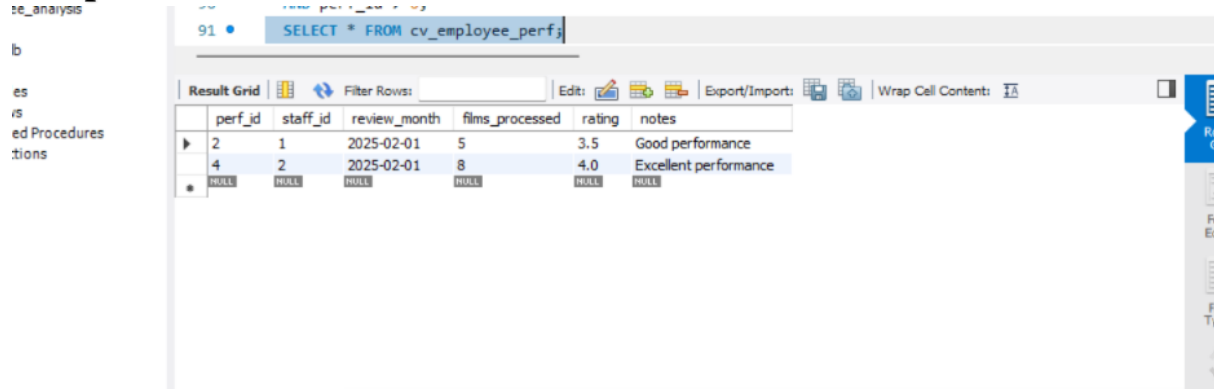
```
AND rating < 2.0
```

```
AND perf_id > 0;
```

EXPLANATION:

The table is recreated and sample records are inserted because it was previously dropped in Q3(b). The DELETE statement removes only the record where films_processed is 0 and rating is below 2.0. The final SELECT verifies that the matching record was deleted while the other records remain.

Output:



The screenshot shows a database interface with a query editor at the top containing the SQL statement: `SELECT * FROM cv_employee_perf;`. Below the editor is a 'Result Grid' displaying the query results. The grid has columns: perf_id, staff_id, review_month, films_processed, rating, and notes. There are three rows of data: a record with perf_id 2, staff_id 1, review_month 2025-02-01, films_processed 5, rating 3.5, and note 'Good performance'; a record with perf_id 4, staff_id 2, review_month 2025-02-01, films_processed 8, rating 4.0, and note 'Excellent performance'; and a row of null values. The interface also includes a 'Filter Rows' button and various export/import options.

perf_id	staff_id	review_month	films_processed	rating	notes
2	1	2025-02-01	5	3.5	Good performance
4	2	2025-02-01	8	4.0	Excellent performance
NULL	NULL	NULL	NULL	NULL	NULL

b) Explain (1–2 lines) what safeguard you should apply before running a DELETE without a WHERE clause in production.

EXPLANATION:

Before executing a DELETE without a WHERE clause in production, **take a database backup and verify the records using a SELECT query**. Use a transaction or other recovery mechanism where possible to prevent accidental data loss.

Section C — Business Filtering & Operators

Q8. Marketing Segment 1 — Multi-condition Filter

Marketing wants a list of customers to target for a loyalty campaign: active customers (active = 1) who live in a district that is either 'California' or 'Texas', and whose customer_id is between 1 and 200. Write a single query using AND, OR, BETWEEN, and IN appropriately (use address/customer join).

SQL Query:

```
SELECT c.customer_id,
```

```

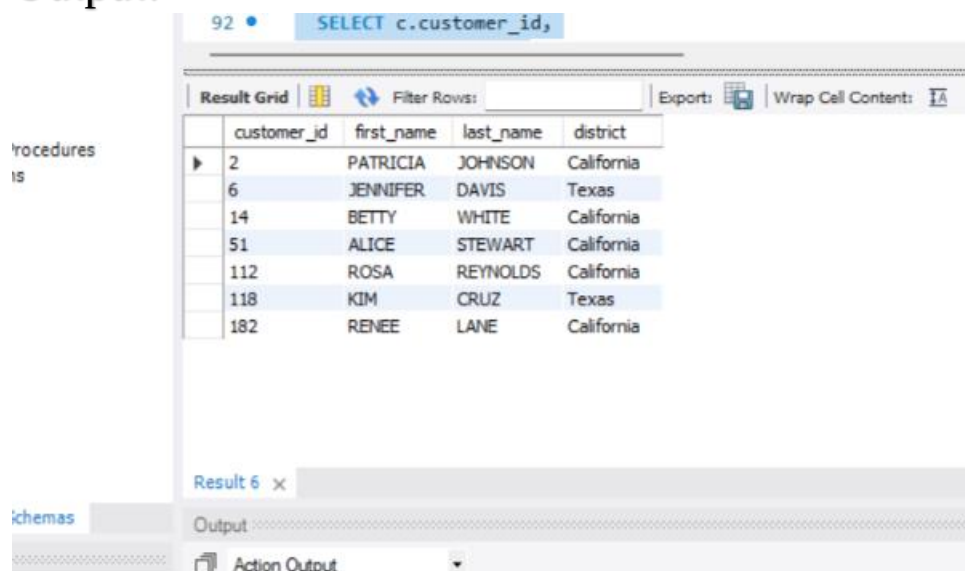
        c.first_name,
        c.last_name,
        a.district
FROM customer c
JOIN address a
    ON c.address_id = a.address_id
WHERE c.active = 1
    AND c.customer_id BETWEEN 1 AND 200
    AND a.district IN ('California', 'Texas');

```

EXPLANATION:

This query joins the customer and address tables using address_id. The WHERE clause selects only **active customers** from **California or Texas** whose customer_id is between **1 and 200**, using AND, OR, IN, and BETWEEN as required.

Output:



The screenshot shows a database interface with a query window containing the SQL query: `SELECT c.customer_id,`. Below the query window is a 'Result Grid' showing the results of the query. The grid has columns for customer_id, first_name, last_name, and district. There are 6 rows of data displayed.

customer_id	first_name	last_name	district
2	PATRICIA	JOHNSON	California
6	JENNIFER	DAVIS	Texas
14	BETTY	WHITE	California
51	ALICE	STEWART	California
112	ROSA	REYNOLDS	California
118	KIM	CRUZ	Texas
182	RENEE	LANE	California

Q9. Marketing Segment 2 — Pattern & NULL Handling

a) Write a query to find all customers whose email contains 'sakilacustomer.org' (use LIKE).

SQL Query:

```
SELECT customer_id,
```

```

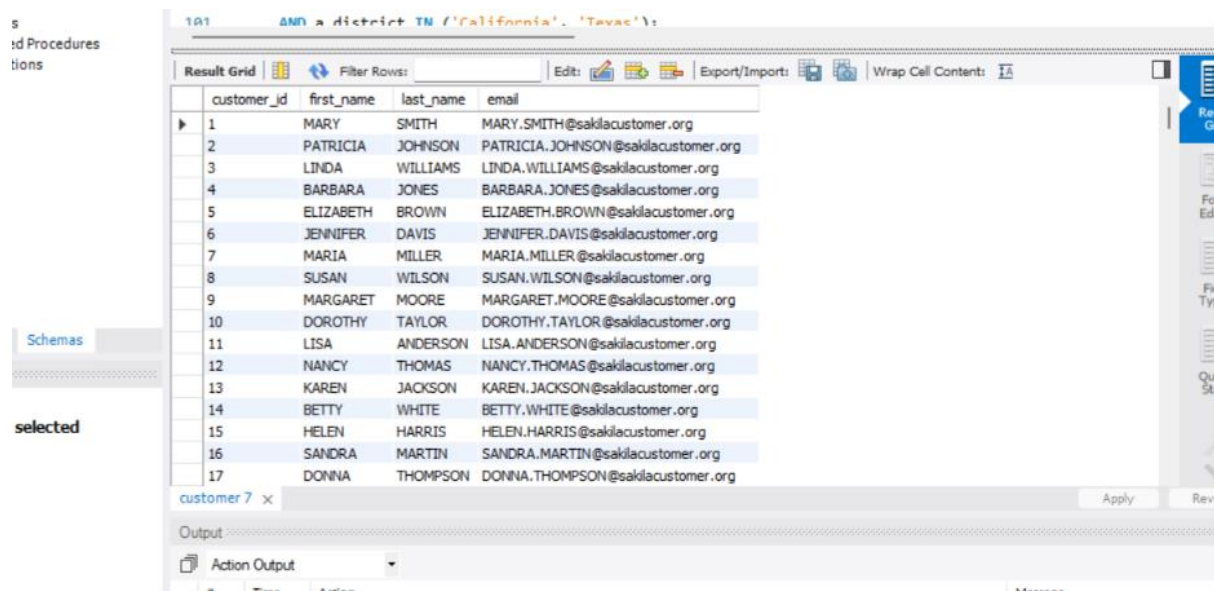
    first_name,
    last_name,
    email
FROM customer
WHERE email LIKE '%sakilacustomer.org%';

```

EXPLANATION:

This query searches the customer table for customers whose **email address contains sakilacustomer.org**. The LIKE operator is used for pattern matching, and the % symbols act as wildcards, meaning any characters can appear before or after the specified text. Thus, only email addresses containing sakilacustomer.org are returned

Output:



customer_id	first_name	last_name	email
1	MARY	SMITH	MARY.SMITH@sakilacustomer.org
2	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org
3	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org
4	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org
5	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org
6	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org
7	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org
8	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org
9	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org
10	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org
11	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.org
12	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org
13	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org
14	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org
15	HELEN	HARRIS	HELEN.HARRIS@sakilacustomer.org
16	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.org
17	DONNA	THOMPSON	DONNA.THOMPSON@sakilacustomer.org

b) Write a query to find all films where the description does not contain the word 'Amazing'.

SQL Query:

```

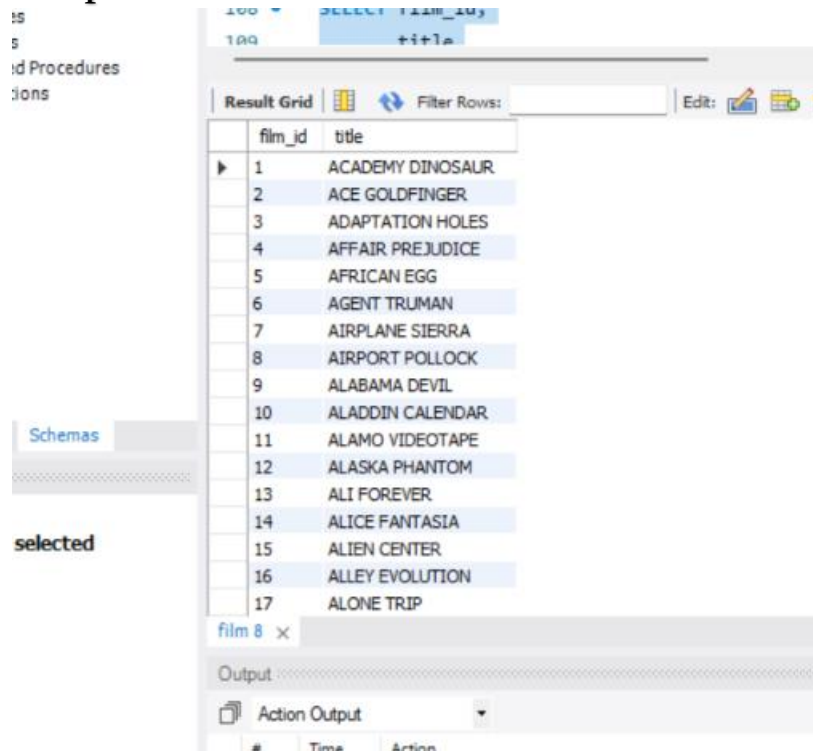
SELECT film_id,
       title
FROM film
WHERE title NOT LIKE '%Amazing%';

```

EXPLANATION:

This query searches the film table and uses NOT LIKE to exclude films whose description contains the word **Amazing**. The % wildcards allow Amazing to appear anywhere within the description, so only films without that word are returned.

Output:



The screenshot shows a database interface with a 'Result Grid' displaying 17 rows of film data. The columns are 'film_id' and 'title'. The films listed are: 1. ACADEMY DINOSAUR, 2. ACE GOLDFINGER, 3. ADAPTATION HOLES, 4. AFFAIR PREJUDICE, 5. AFRICAN EGG, 6. AGENT TRUMAN, 7. AIRPLANE SIERRA, 8. AIRPORT POLLOCK, 9. ALABAMA DEVIL, 10. ALADDIN CALENDAR, 11. ALAMO VIDEOTAPE, 12. ALASKA PHANTOM, 13. ALI FOREVER, 14. ALICE FANTASIA, 15. ALIEN CENTER, 16. ALLEY EVOLUTION, and 17. ALONE TRIP. The interface also shows a 'Schemas' panel on the left and an 'Output' section at the bottom.

film_id	title
1	ACADEMY DINOSAUR
2	ACE GOLDFINGER
3	ADAPTATION HOLES
4	AFFAIR PREJUDICE
5	AFRICAN EGG
6	AGENT TRUMAN
7	AIRPLANE SIERRA
8	AIRPORT POLLOCK
9	ALABAMA DEVIL
10	ALADDIN CALENDAR
11	ALAMO VIDEOTAPE
12	ALASKA PHANTOM
13	ALI FOREVER
14	ALICE FANTASIA
15	ALIEN CENTER
16	ALLEY EVOLUTION
17	ALONE TRIP

- c) Write a query to list all rentals from the rental table where return_date IS NULL (i.e., currently rented out / overdue).

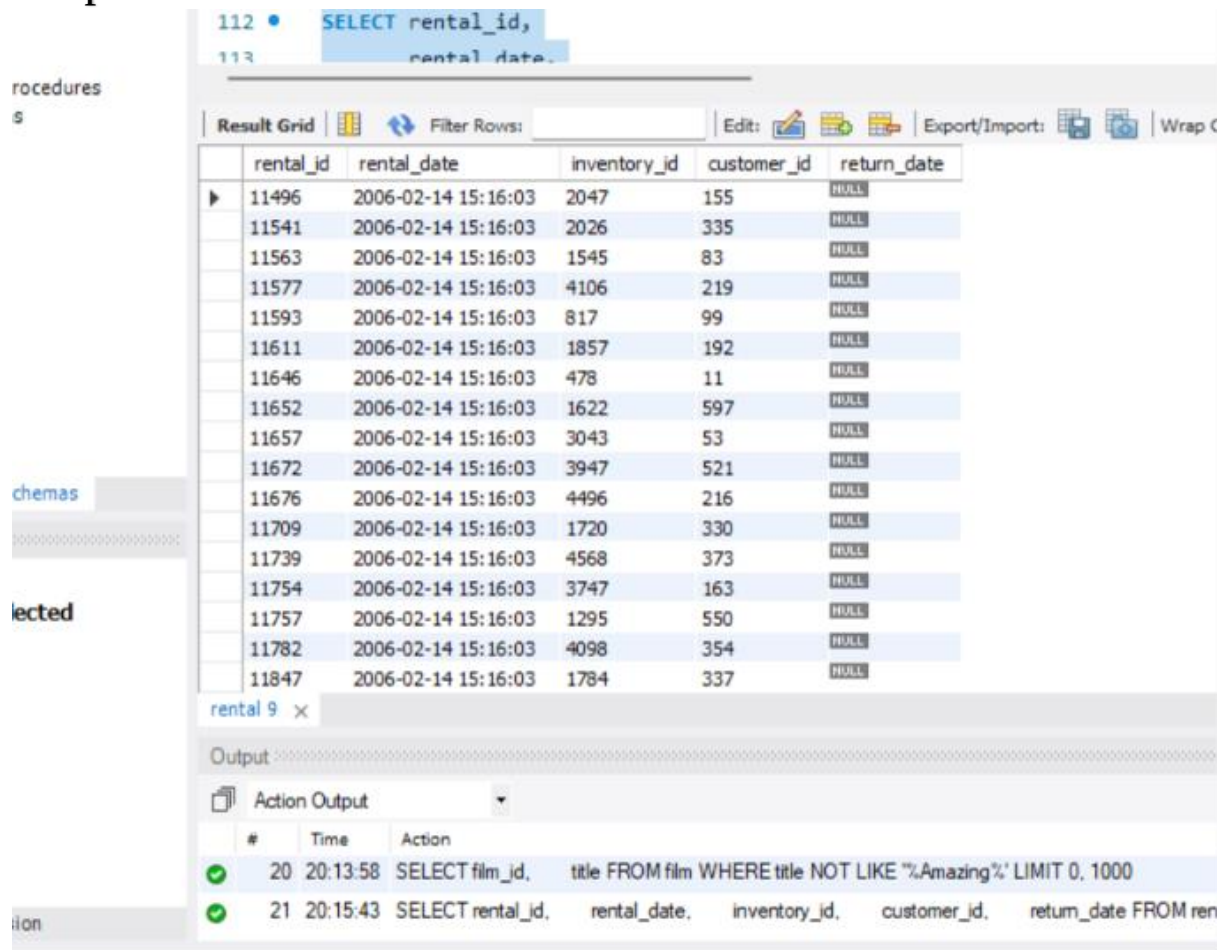
SQL Query:

```
SELECT rental_id,  
       rental_date,  
       inventory_id,  
       customer_id,  
       return_date  
FROM rental  
WHERE return_date IS NULL;
```

EXPLANATION:

This query retrieves rental records where the customer has **not yet returned the rented film**. The IS NULL condition is used because NULL represents a missing or unknown value; it cannot be checked using = NULL.

Output:



The screenshot shows a database management system interface. At the top, a SQL query is entered: `SELECT rental_id, rental_date, inventory_id, customer_id, return_date FROM rental WHERE return_date IS NULL`. Below the query, a 'Result Grid' displays the results of the query. The grid has five columns: `rental_id`, `rental_date`, `inventory_id`, `customer_id`, and `return_date`. The results show 15 rows of data, all with `return_date` set to NULL. Below the result grid, there is an 'Output' section showing the execution of the query. It includes a table with columns: #, Time, and Action. The first action is: `SELECT film_id, title FROM film WHERE title NOT LIKE '%Amazing%' LIMIT 0, 1000` at 20:13:58. The second action is: `SELECT rental_id, rental_date, inventory_id, customer_id, return_date FROM rental` at 20:15:43.

rental_id	rental_date	inventory_id	customer_id	return_date
11496	2006-02-14 15:16:03	2047	155	NULL
11541	2006-02-14 15:16:03	2026	335	NULL
11563	2006-02-14 15:16:03	1545	83	NULL
11577	2006-02-14 15:16:03	4106	219	NULL
11593	2006-02-14 15:16:03	817	99	NULL
11611	2006-02-14 15:16:03	1857	192	NULL
11646	2006-02-14 15:16:03	478	11	NULL
11652	2006-02-14 15:16:03	1622	597	NULL
11657	2006-02-14 15:16:03	3043	53	NULL
11672	2006-02-14 15:16:03	3947	521	NULL
11676	2006-02-14 15:16:03	4496	216	NULL
11709	2006-02-14 15:16:03	1720	330	NULL
11739	2006-02-14 15:16:03	4568	373	NULL
11754	2006-02-14 15:16:03	3747	163	NULL
11757	2006-02-14 15:16:03	1295	550	NULL
11782	2006-02-14 15:16:03	4098	354	NULL
11847	2006-02-14 15:16:03	1784	337	NULL

#	Time	Action
20	20:13:58	SELECT film_id, title FROM film WHERE title NOT LIKE '%Amazing%' LIMIT 0, 1000
21	20:15:43	SELECT rental_id, rental_date, inventory_id, customer_id, return_date FROM rental

Q10. Operator Combination Challenge

Write a query to list film titles that are NOT in the 'Children', 'Music', or 'Travel' categories, have a rental_rate greater than 2.99, and a length NOT BETWEEN 60 and 90 minutes. (Use film, film_category, category.)

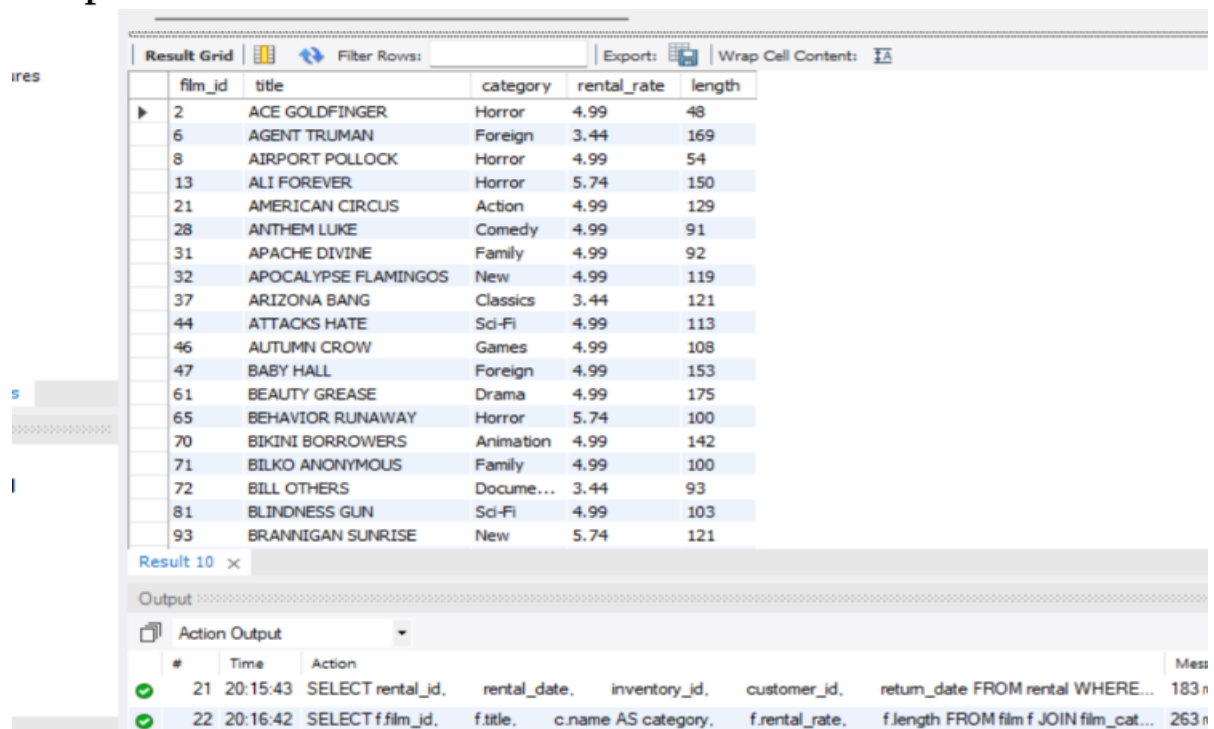
SQL Query:

```
SELECT f.film_id,  
       f.title,  
       c.name AS category,  
       f.rental_rate,  
       f.length  
FROM film f  
JOIN film_category fc  
  ON f.film_id = fc.film_id  
JOIN category c  
  ON fc.category_id = c.category_id  
WHERE c.name NOT IN ('Children', 'Music', 'Travel')  
      AND f.rental_rate > 2.99  
      AND f.length NOT BETWEEN 60 AND 90;
```

EXPLANATION:

The query joins the film, film_category, and category tables to connect each film with its category. NOT IN excludes the **Children, Music, and Travel** categories, > 2.99 selects films with higher rental rates, and NOT BETWEEN 60 AND 90 selects films whose duration is outside the **60–90 minute** range.

Output:



film_id	title	category	rental_rate	length
2	ACE GOLDFINGER	Horror	4.99	48
6	AGENT TRUMAN	Foreign	3.44	169
8	AIRPORT POLLOCK	Horror	4.99	54
13	ALI FOREVER	Horror	5.74	150
21	AMERICAN CIRCUS	Action	4.99	129
28	ANTHEM LUKE	Comedy	4.99	91
31	APACHE DIVINE	Family	4.99	92
32	APOCALYPSE FLAMINGOS	New	4.99	119
37	ARIZONA BANG	Classics	3.44	121
44	ATTACKS HATE	Sci-Fi	4.99	113
46	AUTUMN CROW	Games	4.99	108
47	BABY HALL	Foreign	4.99	153
61	BEAUTY GREASE	Drama	4.99	175
65	BEHAVIOR RUNAWAY	Horror	5.74	100
70	BIKINI BORROWERS	Animation	4.99	142
71	BILKO ANONYMOUS	Family	4.99	100
72	BILL OTHERS	Docume...	3.44	93
81	BLINDNESS GUN	Sci-Fi	4.99	103
93	BRANNIGAN SUNRISE	New	5.74	121

#	Time	Action	Message
21	20:15:43	SELECT rental_id, rental_date, inventory_id, customer_id, return_date FROM rental WHERE...	183 n
22	20:16:42	SELECT f.film_id, f.title, c.name AS category, f.rental_rate, f.length FROM film f JOIN film_cat...	263 n

Section D — Relationship Analysis: All Joins

Q11. Inner Join — Core Report

Write a query to list film title, category name, and language name together for every film (only films that have all three linked) using INNER JOIN.

SQL Query:

```
SELECT f.film_id,  
       f.title,  
       c.name AS category,  
       l.name AS language  
FROM film f  
INNER JOIN film_category fc  
  ON f.film_id = fc.film_id  
INNER JOIN category c  
  ON fc.category_id = c.category_id  
INNER JOIN language l  
  ON f.language_id = l.language_id;
```

EXPLANATION:

This query uses INNER JOIN to connect the film, film_category, category, and language tables. It matches films with their corresponding category and language using their related keys, so only records having matching data in all required tables are displayed.

Output:

ures

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	film_id	title	category	language
▶	19	AMADEUS HOLY	Action	English
	21	AMERICAN CIRCUS	Action	English
	29	ANTITRUST TOMATOES	Action	English
	38	ARK RIDGEMONT	Action	English
	56	BAREFOOT MANCHURIAN	Action	English
	67	BERETS AGENT	Action	English
	97	BRIDE INTRIGUE	Action	English
	105	BULL SHAWSHANK	Action	English
	111	CADDYSHACK JEDI	Action	English
	115	CAMPUS REMEMBER	Action	English
	126	CASUALTIES ENCINO	Action	English
	130	CELEBRITY HORN	Action	English
	162	CLUELESS BUCKET	Action	English
	194	CROW GREASE	Action	English
	205	DANCES NONE	Action	English
	210	DARKO DORADO	Action	English
	212	DARN FORRESTER	Action	English
	229	DEVIL DESIRE	Action	English

Result 11 x

Output

Action Output

#	Time	Action	Messi
✓	22:10:16:42	SELECT f.film_id, f.title, c.name AS category, f.rental_rate, f.length FROM film f JOIN film_cat...	263 r

d

Q12. Left & Right Join — Gap Analysis

a) Using LEFT JOIN, list every customer along with their rental count — customers with zero rentals must still appear with a count of 0.

SQL Query:

```
SELECT c.customer_id,
       c.first_name,
       c.last_name,
       COUNT(r.rental_id) AS rental_count
FROM customer c
LEFT JOIN rental r
  ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

EXPLANATION:

The LEFT JOIN keeps **every customer**, even if they have no matching rental record. COUNT(r.rental_id) counts only actual rentals, so customers with no rentals are displayed with a count of **0**.

Output:

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	customer_id	first_name	last_name	rental_count
▶	1	MARY	SMITH	32
	2	PATRICIA	JOHNSON	27
	3	LINDA	WILLIAMS	26
	4	BARBARA	JONES	22
	5	ELIZABETH	BROWN	38
	6	JENNIFER	DAVIS	28
	7	MARIA	MILLER	33
	8	SUSAN	WILSON	24
	9	MARGARET	MOORE	23
	10	DOROTHY	TAYLOR	25
	11	LISA	ANDERSON	24
	12	NANCY	THOMAS	28
	13	KAREN	JACKSON	27
	14	BETTY	WHITE	28
	15	HELEN	HARRIS	32
	16	SANDRA	MARTIN	28
	17	DONNA	THOMPSON	21
	18	CAROL	GARCIA	22

Result 12

×

Output

Action Output

#

Time

Action

Message

✓

23

20:18:59

SELECT f.film_id, f.title, c.name AS category, l.name AS language FROM film f INNER JOIN film_...

1000 ro

✓

24

20:20:25

SELECT c.customer_id, c.first_name, c.last_name, COUNT(r.rental_id) AS rental_count FROM c...

599 row

b) Using RIGHT JOIN (rewrite the same logic from the opposite table order), produce the same result — write it starting FROM rental ... RIGHT JOIN customer.

SQL Query:

```
SELECT r.rental_id,
       r.rental_date,
       r.customer_id,
       c.first_name,
       c.last_name
FROM rental r
RIGHT JOIN customer c
  ON r.customer_id = c.customer_id;
```

EXPLANATION:

The RIGHT JOIN keeps **all rows from the customer table**, including customers without rentals. Since the customer table is on the right side of the join, every customer is preserved and COUNT(r.rental_id) gives **0** when no rental exists.

Output:

```

156 FROM rental r
157 RIGHT JOIN customer c
158 ON r.customer_id = c.customer_id;

```

res

Result Grid

	rental_id	rental_date	customer_id	first_name	last_name
▶	76	2005-05-25 11:30:37	1	MARY	SMITH
	573	2005-05-28 10:35:23	1	MARY	SMITH
	1185	2005-06-15 00:54:12	1	MARY	SMITH
	1422	2005-06-15 18:02:53	1	MARY	SMITH
	1476	2005-06-15 21:08:46	1	MARY	SMITH
	1725	2005-06-16 15:18:57	1	MARY	SMITH
	2308	2005-06-18 08:41:48	1	MARY	SMITH
	2363	2005-06-18 13:33:59	1	MARY	SMITH
	3284	2005-06-21 06:24:45	1	MARY	SMITH
	4526	2005-07-08 03:17:05	1	MARY	SMITH
	4611	2005-07-08 07:33:56	1	MARY	SMITH
	5244	2005-07-09 13:24:07	1	MARY	SMITH
	5326	2005-07-09 16:38:01	1	MARY	SMITH
	6163	2005-07-11 10:13:46	1	MARY	SMITH
	7273	2005-07-27 11:31:22	1	MARY	SMITH
	7841	2005-07-28 09:04:45	1	MARY	SMITH
	8033	2005-07-28 16:18:23	1	MARY	SMITH
	8074	2005-07-28 17:33:39	1	MARY	SMITH

Result 13 x

Output

Action Output

#	Time	Action
✓ 24	20:20:25	SELECT c.customer_id, c.first_name, c.last_name, COUNT(r.rental_id) AS
✓ 25	20:21:30	SELECT r.rental_id, r.rental_date, r.customer_id, c.first_name, c.last_n

Q13. Full Outer Join Simulation

MySQL has no native FULL OUTER JOIN. CineVault wants a report of every actor and every film they may or may not be linked to, including actors with no films and films with no actors. Write a query using LEFT JOIN ... UNION ... RIGHT JOIN (or UNION of two LEFT JOINS) on actor, film_actor, and film to simulate this.

SQL Query:

```

SELECT
  a.actor_id,
  a.first_name,

```

```
    a.last_name,  
    fa.film_id,  
    f.title  
FROM actor a  
LEFT JOIN film_actor fa  
    ON a.actor_id = fa.actor_id  
LEFT JOIN film f  
    ON fa.film_id = f.film_id
```

UNION

```
SELECT  
    a.actor_id,  
    a.first_name,  
    a.last_name,  
    fa.film_id,  
    f.title  
FROM actor a  
RIGHT JOIN film_actor fa  
    ON a.actor_id = fa.actor_id  
LEFT JOIN film f  
    ON fa.film_id = f.film_id;
```

EXPLANATION:

MySQL does not directly support FULL OUTER JOIN, so this query combines a LEFT JOIN and a RIGHT JOIN using UNION. The first part keeps **all actors**, even those without films, while the second part keeps **all films**, even those without actors. UNION combines both results and removes duplicate rows.

Output:

```

182 LEFT JOIN film f
183 ON fa.film_id = f.film_id;

```

Result Grid					
Filter Rows:					
Export: Wrap Cell Content:					
	actor_id	first_name	last_name	film_id	title
▶	1	PENELOPE	GUINESS	1	ACADEMY DINOSAUR
	1	PENELOPE	GUINESS	23	ANACONDA CONFESSIONS
	1	PENELOPE	GUINESS	25	ANGELS LIFE
	1	PENELOPE	GUINESS	106	BULWORTH COMMANDMENTS
	1	PENELOPE	GUINESS	140	CHEAPER CLYDE
	1	PENELOPE	GUINESS	166	COLOR PHILADELPHIA
	1	PENELOPE	GUINESS	277	ELEPHANT TROJAN
	1	PENELOPE	GUINESS	361	GLEAMING JAWBREAKER
	1	PENELOPE	GUINESS	438	HUMAN GRAFFITI
	1	PENELOPE	GUINESS	499	KING EVOLUTION
	1	PENELOPE	GUINESS	506	LADY STAGE
	1	PENELOPE	GUINESS	509	LANGUAGE COWBOY
	1	PENELOPE	GUINESS	605	MULHOLLAND BEAST
	1	PENELOPE	GUINESS	635	OKLAHOMA JUMANJI
	1	PENELOPE	GUINESS	749	RULES HUMAN
	1	PENELOPE	GUINESS	832	SPLASH GUMP
	1	PENELOPE	GUINESS	939	VERTIGO NORTHWEST
	1	PENELOPE	GUINESS	970	WESTWARD SEABISCUIT

Result 14 x

Output

Action Output

#	Time	Action
✓ 25	20:21:30	SELECT r.rental_id, r.rental_date, r.customer_id, c.first_name, c.l
✓ 26	20:22:30	SELECT a.actor_id, a.first_name, a.last_name, fa.film_id, f.title FROM

Q14. Self Join — Peer Comparison

HR wants to compare staff members who work at the same store. Write a self-join query on staff that lists pairs of staff members (s1.first_name, s2.first_name) working at the same store_id, excluding a staff member being paired with themselves.

SQL Query:

```

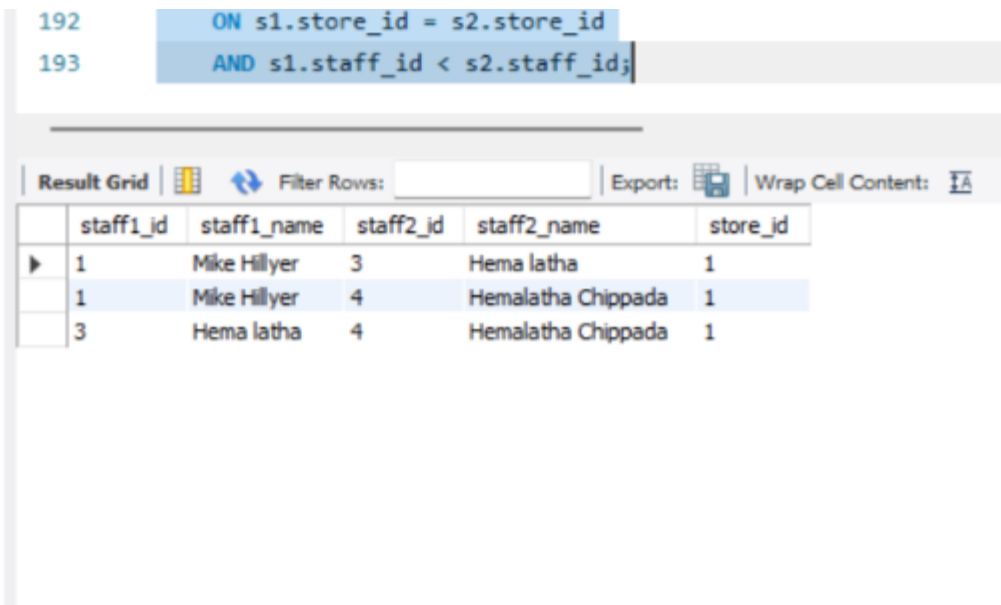
SELECT
    s1.staff_id AS staff1_id,
    CONCAT(s1.first_name, ' ', s1.last_name) AS staff1_name,
    s2.staff_id AS staff2_id,
    CONCAT(s2.first_name, ' ', s2.last_name) AS staff2_name,
    s1.store_id
FROM staff s1
JOIN staff s2
    ON s1.store_id = s2.store_id
    AND s1.staff_id < s2.staff_id;

```

EXPLANATION:

This query uses a **self join**, meaning the staff table is joined with itself using two aliases, s1 and s2. The store_id condition finds staff members working at the same store, while s1.staff_id < s2.staff_id prevents a staff member from being paired with themselves and avoids duplicate reverse pairs.

Output:



```
192      ON s1.store_id = s2.store_id
193      AND s1.staff_id < s2.staff_id;
```

	staff1_id	staff1_name	staff2_id	staff2_name	store_id
▶	1	Mike Hillyer	3	Hema latha	1
	1	Mike Hillyer	4	Hemalatha Chippada	1
	3	Hema latha	4	Hemalatha Chippada	1

Q15. Cross Join — Scenario Planning

Marketing is planning a promotion matrix: every store combined with every film category, to decide which categories to promote at which store. Write a CROSS JOIN between store and category to generate this combination list. (Add a note on why CROSS JOIN should be used cautiously on large tables.)

SQL Query:

```
SELECT s.store_id,
       c.category_id,
       c.name AS category_name
FROM store s
CROSS JOIN category c;
```

EXPLANATION:

This query uses CROSS JOIN to create **every possible combination of stores and film categories**. Since there is no join condition, each store is matched with every category. CROSS JOIN should be used carefully on large tables because it can produce a very large number of rows and consume significant processing resources.

Output:

The screenshot shows a database query editor with a query window and a results window.

Query Window:

```
192      ON s1.store_id = s2.store_id
193      AND s1.staff_id < s2.staff_id;
194      SELECT
195      s.store_id,
```

Results Window:

Result Grid | Filter Rows: | Export: | Wr

	store_id	category_id	category_name
▶	2	1	Action
	1	1	Action
	2	2	Animation
	1	2	Animation
	2	3	Children
	1	3	Children
	2	4	Classics
	1	4	Classics
	2	5	Comedy
	1	5	Comedy
	2	6	Documentary
	1	6	Documentary
	2	7	Drama
	1	7	Drama
	2	8	Family
	1	8	Family
	2	9	Foreign
	1	9	Foreign

Result 16 x

Output:

Action Output

#	Time	Action
✓ 27	20:23:43	SELECT s1.staff_id AS staff1_id, CONCAT(s1.first_r
✓ 28	20:25:07	SELECT s.store_id, c.category_id, c.name AS cat

Session

Section E — Aggregation, Grouping & Output-Based Questions

Q16. Revenue Report — Grouped

Finance wants monthly revenue trends.

a) Write a query using GROUP BY to show total revenue (SUM(amount)) per staff_id from payment.

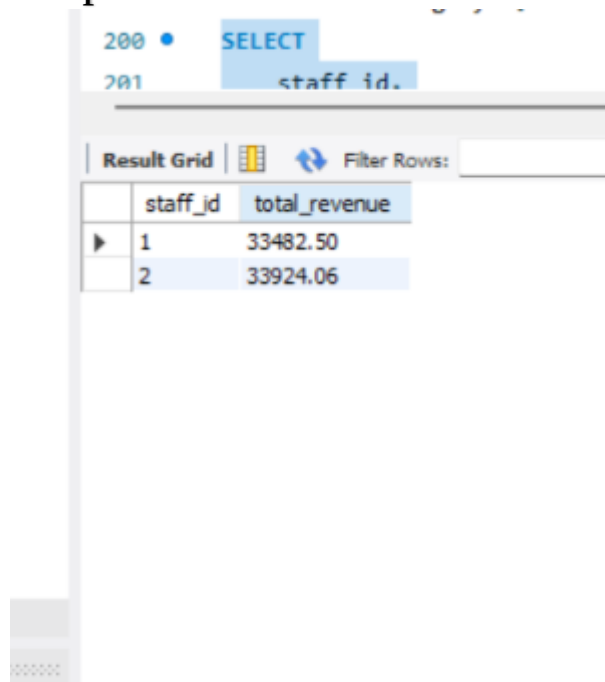
SQL Query:

```
SELECT
    staff_id,
    SUM(amount) AS total_revenue
FROM payment
GROUP BY staff_id
HAVING SUM(amount) > 15000;
```

EXPLANATION:

This query groups all payment records according to staff_id. The SUM(amount) function then calculates the **total revenue generated by each staff member**, giving one result row for each staff ID.

Output:



The screenshot shows a SQL query editor with a query window at the top and a results window below. The query window contains the following SQL query:

```
SELECT
    staff_id,
    SUM(amount) AS total_revenue
FROM payment
GROUP BY staff_id
HAVING SUM(amount) > 15000;
```

The results window displays a table with two columns: staff_id and total_revenue. The table contains two rows of data:

staff_id	total_revenue
1	33482.50
2	33924.06

b) Extend it with HAVING to show only staff whose total revenue exceeds 15000.

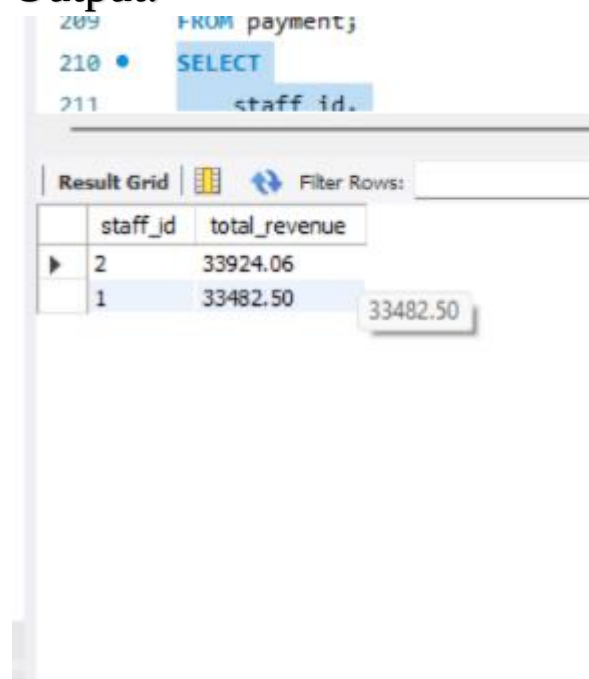
SQL Query:

```
SELECT
    staff_id,
    SUM(amount) AS total_revenue
FROM payment
GROUP BY staff_id
HAVING SUM(amount) > 15000
ORDER BY total_revenue DESC;
```

EXPLANATION:

The query groups payment records by `staff_id` and calculates the total revenue for each staff member using `SUM(amount)`. The `HAVING` clause then filters the grouped results and displays only staff whose total revenue is **greater than 15,000**.

Output:



The screenshot shows a SQL query editor with the following query:

```
209 FROM payment;
210 SELECT
211     staff_id,
```

Below the query editor is a 'Result Grid' with the following data:

	staff_id	total_revenue
▶	2	33924.06
	1	33482.50

A tooltip is visible over the value 33482.50, displaying the number 33482.50.

c) Now write a version without grouping — a single query showing the overall company-wide total revenue and the single highest individual payment amount, side by side (use aggregate functions without `GROUP BY`).

SQL Query:

```

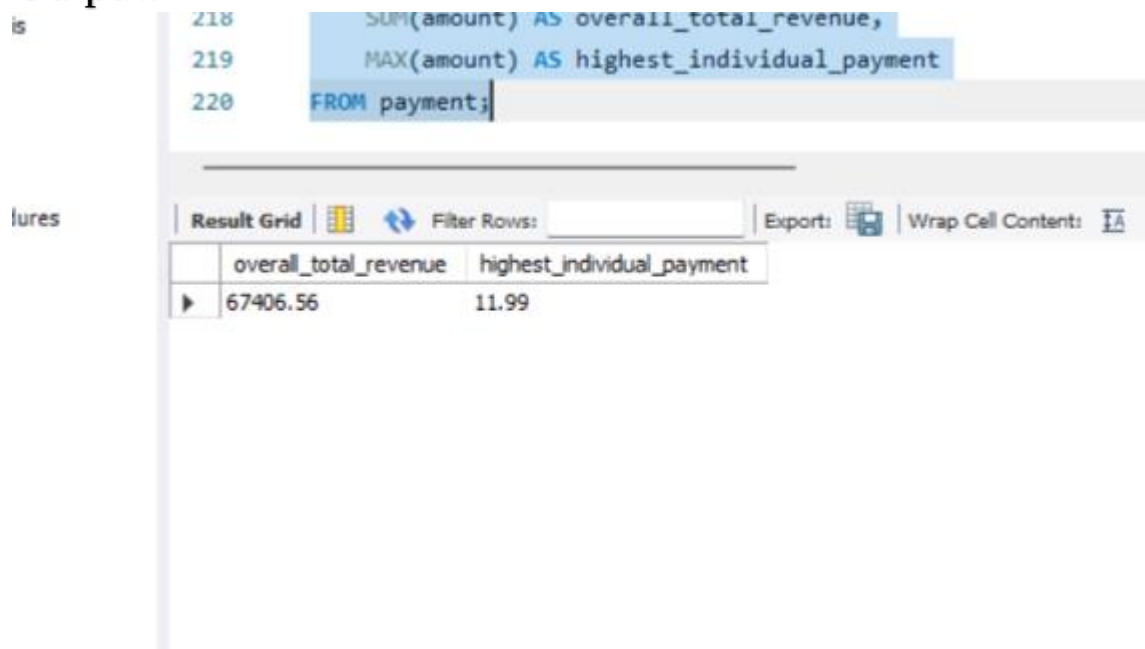
SELECT
    SUM(amount) AS overall_total_revenue,
    MAX(amount) AS highest_individual_payment
FROM payment;

```

EXPLANATION:

This query calculates the **overall revenue** using SUM(amount) and finds the **highest individual payment** using MAX(amount). Since there is no GROUP BY clause, the aggregate functions operate on the entire payment table and return a single row.

Output:



Q17. Output-Based Question — Reverse Engineer the Query

You are shown the following result set from a query run on sakila:

category_name	total_films	avg_rental_rate
Action	64	3.28
Sports	74	3.13

category_name	total_films	avg_rental_rate
Sci-Fi	61	2.99

The result is sorted by total_films descending, and only shows categories with more than 60 films.

Write the exact MySQL query that would produce this output.
(Tables: film, film_category, category.)

SQL Query:

```
SELECT
    c.name AS category_name,
    COUNT(f.film_id) AS film_count,
    ROUND(AVG(f.rental_rate), 2) AS average_rental_rate
FROM category c
JOIN film_category fc
    ON c.category_id = fc.category_id
JOIN film f
    ON fc.film_id = f.film_id
GROUP BY c.category_id, c.name
HAVING COUNT(f.film_id) > 60
ORDER BY film_count DESC;
```

EXPLANATION:

The query joins the film, film_category, and category tables to associate each film with its category. COUNT() calculates the number of films and AVG() calculates the average rental rate for each category. HAVING keeps only categories with **more than 60 films**, while ORDER BY sorts them from the highest number of films to the lowest.

Output:

```

228 JOIN film f
229     ON fc.film_id = f.film_id
230 GROUP BY c.category_id, c.name
231 HAVING COUNT(f.film_id) > 60
232 ORDER BY film_count DESC;

```

es

Result Grid | Filter Rows: | Export: | Wrap

	category_name	film_count	average_rental_rate
▶	Sports	74	3.24
	Foreign	73	3.17
	Family	69	2.88
	Documentary	68	2.76
	Animation	66	2.89
	Action	64	2.70
	New	63	3.19
	Drama	62	3.06
	Games	61	3.30
	Sci-Fi	61	3.32

Q18. Output-Based Question — Rank the Customers

You are shown this result:

customer_id	first_name	total_spent	spend_rank
137	Karl	221.55	1
526	Eleanor	216.54	2
148	Clara	195.58	3

The `spend_rank` column ranks customers by their total payments, highest first, with no gaps in ranking even if totals tie.

Write the query — you must use a window function (`RANK()` or `DENSE_RANK()`), not a subquery-only approach, to generate `spend_rank`.

SQL Query:

```
SELECT
    customer_id,
    first_name,
    last_name,
    total_spending,
    RANK() OVER (ORDER BY total_spending DESC) AS spending_rank
FROM (
    SELECT
        c.customer_id,
        c.first_name,
        c.last_name,
        SUM(p.amount) AS total_spending
    FROM customer c
    JOIN payment p
        ON c.customer_id = p.customer_id
    GROUP BY c.customer_id, c.first_name, c.last_name
) AS customer_spending
ORDER BY spending_rank;
```

EXPLANATION:

The inner query calculates the **total amount spent by each customer** using SUM(amount). The outer query uses the DENSE_RANK() window function to rank customers from the highest total spending to the lowest. DENSE_RANK() gives the same rank to customers with equal spending **without leaving gaps in the ranking**.

Output:

244

SUM(p.amount) AS total_spending

245

FROM customer c

246

JOIN payment p

Result Grid

Filter Rows:

Export:

Wrap Cell Contents:

	customer_id	first_name	last_name	total_spending	spending_rank
▶	526	KARL	SEAL	221.55	1
	148	ELEANOR	HUNT	216.54	2
	144	CLARA	SHAW	195.58	3
	137	RHONDA	KENNEDY	194.61	4
	178	MARION	SNYDER	194.61	4
	459	TOMMY	COLLAZO	186.62	6
	469	WESLEY	BULL	177.60	7
	468	TIM	CARY	175.61	8
	236	MARCIA	DEAN	175.58	9
	181	ANA	BRADLEY	174.66	10
	176	JUNE	CARROLL	173.63	11
	50	DIANE	COLLINS	169.65	12
	259	LENA	JENSEN	168.68	13
	522	ARNOLD	HAVENS	167.67	14
	410	CURTIS	IRBY	167.62	15
	403	MIKE	WAY	166.65	16
	295	DAISY	BATES	162.62	17
	209	TONYA	CHAPMAN	161.68	18
	373	LOUIS	LEONE	161.65	19
	470	GORDON	ALLARD	160.68	20
	187	BRITTANY	RILEY	159.72	21
	550	GUY	BROWNLEE	159.68	22
	462	WARREN	SHERROD	159.67	23

Result 22

×

Output

Action Output

▼

Section F — Advanced Analytics: Subqueries & Window Functions

Q19. Subqueries — Nested & Correlated

a) Nested subquery: Write a query to find all films whose rental_rate is higher than the average rental rate of their own category (hint: correlated subquery referencing the outer film's category_id).

SQL Query:

```
SELECT
    f.film_id,
    f.title,
```

```
    c.name AS category_name,  
    f.rental_rate  
FROM film f  
JOIN film_category fc  
    ON f.film_id = fc.film_id  
JOIN category c  
    ON fc.category_id = c.category_id  
WHERE f.rental_rate > (  
    SELECT AVG(f2.rental_rate)  
    FROM film f2  
    JOIN film_category fc2  
        ON f2.film_id = fc2.film_id  
    WHERE fc2.category_id = fc.category_id  
)  
ORDER BY f.rental_rate DESC;
```

EXPLANATION:

The outer query selects films and their rental rates, while the subquery calculates the **average rental rate for the film's category**. The subquery is correlated because `fc2.category_id = fc.category_id` connects it to the category of the current outer film. Only films whose rental rate is **higher than their own category's average** are returned.

Output:

260 ON fc.category_id = c.category_id

Result Grid Filter Rows: Export: Wrap Cell Content:

	film_id	title	category_name	rental_rate
▶	327	FOOL MOCKINGBIRD	Action	5.74
	134	CHAMPION FLATLINERS	Animation	5.74
	703	PUNK DIVORCE	Animation	5.74
	859	SUGAR WONKA	Animation	5.74
	485	JERSEY SASSY	Children	5.74
	515	LEGALLY SECRETARY	Children	5.74
	755	SABRINA MIDNIGHT	Children	5.74
	883	TEQUILA PAST	Children	5.74
	918	TWISTED PIRATES	Children	5.74
	955	WALLS ARTIST	Children	5.74
	131	CENTER DINOSAUR	Classics	5.74
	469	IRON MOON	Classics	5.74
	611	MUSKETEERS WAIT	Classics	5.74
	950	VOLUME HOUSE	Classics	5.74
	265	DYING MAKER	Comedy	5.74
	680	PINOCCHIO SIMON	Comedy	5.74
	814	SNATCH SLIPPER	Comedy	5.74
	871	SWEDEN SHINING	Comedy	5.74
	336	FRENCH HOLIDAY	Documentary	5.74
	670	PELICAN COMFORTS	Documentary	5.74
	713	RAINBOW SHOCK	Documentary	5.74
	776	SECRET GROUNDHOG	Documentary	5.74
	662	PATHS CONTROL	Drama	5.74

Result 23 x

Output

Action Output

b) EXISTS subquery: Write a query to list customers who have never made a payment (use NOT EXISTS).

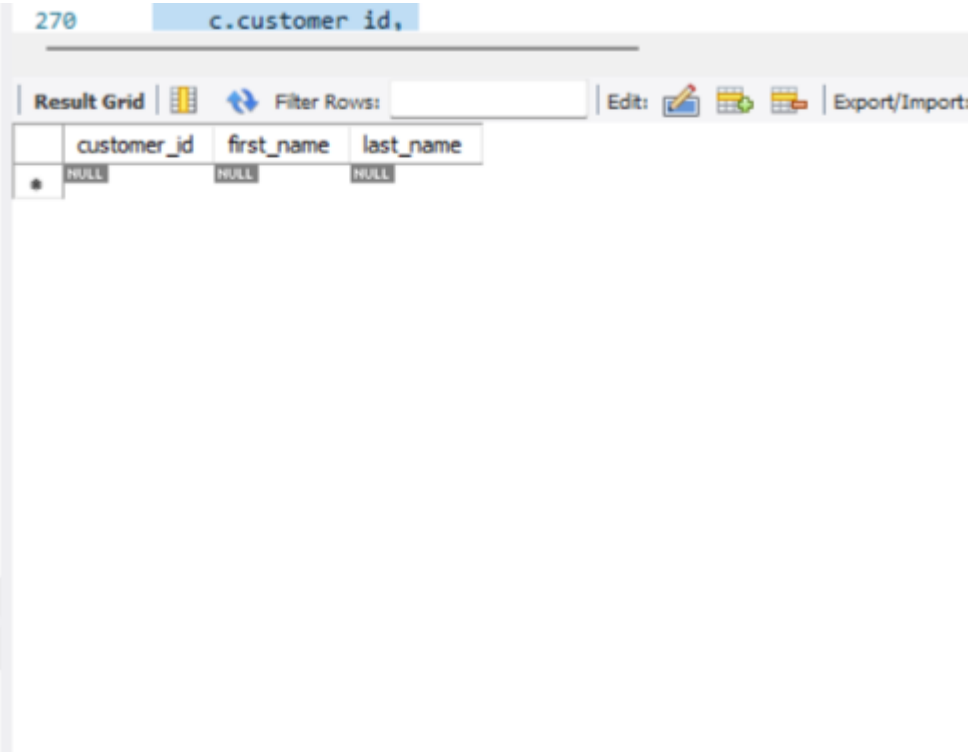
SQL Query:

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name
FROM customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM payment p
    WHERE p.customer_id = c.customer_id
);
```

EXPLANATION:

The outer query checks each customer in the customer table. The NOT EXISTS subquery searches for a payment belonging to that customer; if no matching payment is found, the customer is included in the result. This identifies customers who have **never made a payment**.

Output:



The screenshot shows a database query result grid. The title bar indicates 270 rows and the column c.customer_id. The grid has columns customer_id, first_name, and last_name. A single row is visible with NULL values for all three columns. The interface includes a 'Result Grid' tab, a 'Filter Rows' input, and buttons for 'Edit', 'Export/Import', and 'Export/Import'.

customer_id	first_name	last_name
NULL	NULL	NULL

c) Briefly explain (1–2 lines) the performance difference between using IN with a subquery versus EXISTS when the subquery returns a large number of rows.

EXPLANATION:

When a subquery returns a large number of rows, EXISTS can be more efficient because it can stop searching once a matching row is found, while IN may need to compare against the returned set of values. However, the actual performance depends on the **query optimizer, indexes, and execution plan**.

Q20. Window Functions — Store-Level Analysis

Operations wants to identify, for each store, the top 3 films by total rental count (a “Top 3 per Store” leaderboard).

a) Write a query using ROW_NUMBER() (or RANK()) partitioned by store_id and ordered by rental count descending, then filter to only the top 3 per store (use a CTE or subquery, since you cannot filter directly on a window function in the same SELECT).

SQL Query:

```
-- Rank films within each store by total rental count and return the top 3
WITH film_rental_counts AS (
SELECT
i.store_id,
i.film_id,
f.title,
COUNT(r.rental_id) AS rental_count
FROM inventory i
INNER JOIN film f
ON i.film_id = f.film_id
LEFT JOIN rental r
ON i.inventory_id = r.inventory_id
GROUP BY
i.store_id,
i.film_id,
f.title
),
ranked_films AS (
SELECT

store_id,
film_id,
title,
rental_count,
ROW_NUMBER() OVER (
PARTITION BY store_id
ORDER BY rental_count DESC, film_id
) AS film_rank
FROM film_rental_counts
)
SELECT
store_id,
```

```

film_id,
title,
rental_count,
film_rank
FROM ranked_films
WHERE film_rank <= 3
ORDER BY store_id, film_rank;

```

EXPLANATION:

The first CTE, `film_rental_counts`, calculates the total number of rentals for each film in each store. The second CTE, `ranked_films`, uses `ROW_NUMBER()` to rank films separately within each store based on rental count in descending order. The final query filters the ranking to display only the **top 3 films per store**.

Output:

```

362 WHERE film_rank <= 3
363 ORDER BY store_id, film_rank;

```

	store_id	film_id	title	rental_count	film_rank
▶	1	535	LOVE SUICIDES	20	1
	1	55	BARBARELLA STREETCAR	18	2
	1	489	JUGGLER HARDLY	18	3
	2	450	IDOLS SNATCHERS	20	1
	2	228	DETECTIVE VISION	19	2
	2	396	HANGING DEEP	19	3

b) In one line, explain the difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` if two films in the same store had the exact same rental count.

EXPLANATION:

ROW_NUMBER(), RANK(), and DENSE_RANK() are window functions used to assign rankings to rows based on a specified order.

- **ROW_NUMBER()** → Gives a unique sequential number to every row, even when values are tied.
- **RANK()** → Gives the same rank to tied values, but skips the next rank.
- **DENSE_RANK()** → Gives the same rank to tied values without skipping any rank.

Example: If two films are tied for 1st place:

- ROW_NUMBER() → 1, 2, 3
- RANK() → 1, 1, 3
- DENSE_RANK() → 1, 1, 2

Bonus

Write a single query combining a CTE, a JOIN, and a window function to show each customer's rental history along with a running total (SUM() OVER) of their spending, ordered by payment date.

SQL Query:

```
WITH customer_payments AS (  
  SELECT  
    c.customer_id,  
    c.first_name,  
    c.last_name,  
    p.payment_id,  
    p.payment_date,  
    p.amount  
  FROM customer c  
  INNER JOIN payment p  
    ON c.customer_id = p.customer_id  
)  
SELECT  
  customer_id,  
  first_name,
```

```
last_name,  
payment_id,  
payment_date,  
amount,  
SUM(amount) OVER (  
    PARTITION BY customer_id  
    ORDER BY payment_date, payment_id  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT  
ROW  
    ) AS running_total  
FROM customer_payments  
ORDER BY customer_id, payment_date, payment_id;
```

EXPLANATION:

The CTE `customer_rentals` combines customer, rental, and payment information using `INNER JOIN`. It collects each customer's rental history along with the corresponding payment date and amount.

The `SUM(amount) OVER()` window function calculates a **running total of spending for each customer**. `PARTITION BY customer_id` keeps the calculation separate for each customer, while `ORDER BY payment_date` calculates the total in payment-date order.

Finally, the results are ordered by customer and payment date so that each customer's rental history and spending progression can be easily viewed.

Output:

ures

is

f

Result Grid							
		Filter Rows:		Export:	Wrap Cell Contents:	Fetch rows:	
	customer_id	first_name	last_name	payment_id	payment_date	amount	running_total
▶	1	MARY	SMITH	1	2005-05-25 11:30:37	2.99	2.99
	1	MARY	SMITH	2	2005-05-28 10:35:23	0.99	3.98
	1	MARY	SMITH	3	2005-06-15 00:54:12	5.99	9.97
	1	MARY	SMITH	4	2005-06-15 18:02:53	0.99	10.96
	1	MARY	SMITH	5	2005-06-15 21:08:46	9.99	20.95
	1	MARY	SMITH	6	2005-06-16 15:18:57	4.99	25.94
	1	MARY	SMITH	7	2005-06-18 08:41:48	4.99	30.93
	1	MARY	SMITH	8	2005-06-18 13:33:59	0.99	31.92
	1	MARY	SMITH	9	2005-06-21 06:24:45	3.99	35.91
	1	MARY	SMITH	10	2005-07-08 03:17:05	5.99	41.90
	1	MARY	SMITH	11	2005-07-08 07:33:56	5.99	47.89
	1	MARY	SMITH	12	2005-07-09 13:24:07	4.99	52.88
	1	MARY	SMITH	13	2005-07-09 16:38:01	4.99	57.87
	1	MARY	SMITH	14	2005-07-11 10:13:46	7.99	65.86
	1	MARY	SMITH	15	2005-07-27 11:31:22	2.99	68.85
	1	MARY	SMITH	16	2005-07-28 09:04:45	4.99	73.84
	1	MARY	SMITH	17	2005-07-28 16:18:23	4.99	78.83
	1	MARY	SMITH	18	2005-07-28 17:33:39	0.99	79.82
	1	MARY	SMITH	19	2005-07-28 19:20:07	0.99	80.81
	1	MARY	SMITH	20	2005-07-29 03:58:49	2.99	83.80
	1	MARY	SMITH	21	2005-07-31 02:42:18	2.99	86.79
	1	MARY	SMITH	22	2005-08-01 08:51:04	4.99	91.78
	1	MARY	SMITH	23	2005-08-02 15:36:52	3.99	95.77

Result 28 x

Output

Action Output